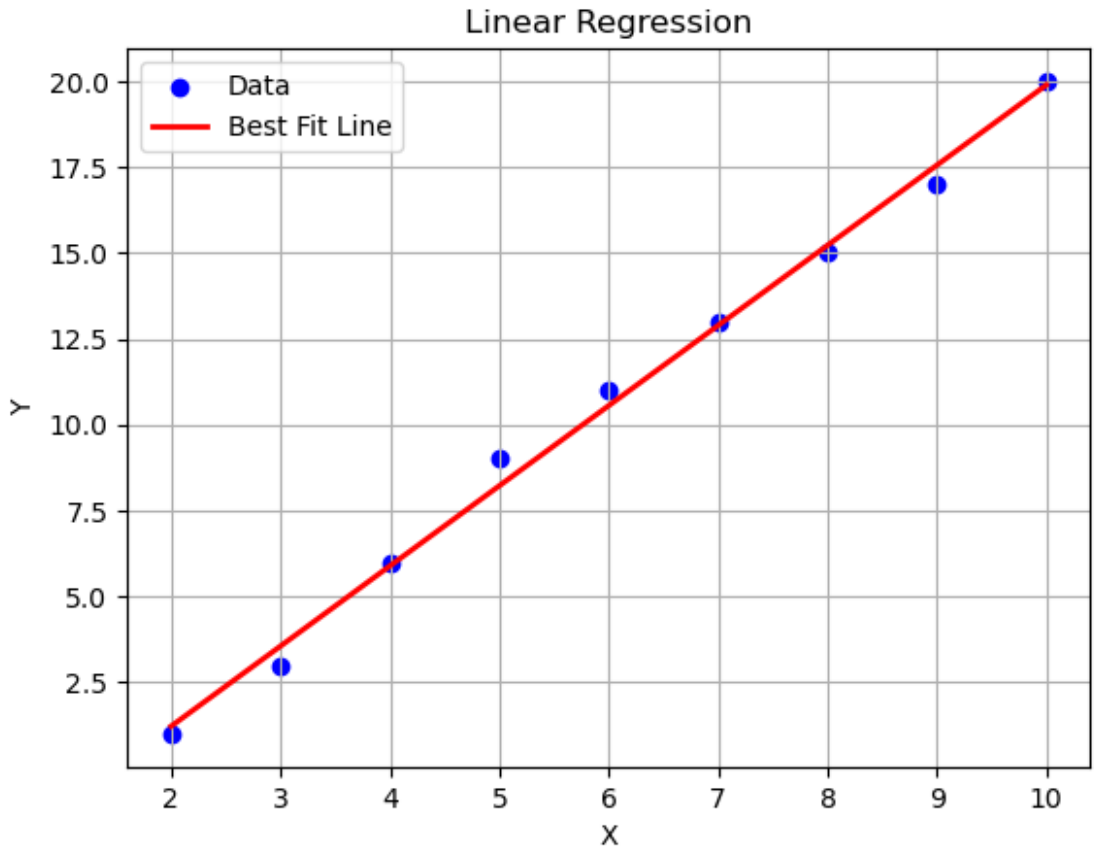
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

X=[2,3,4,5,6,7,8,9,10]
y=[1,3,6,9,11,13,15,17,20]

import numpy as np
import matplotlib.pyplot as plt
X = np.array([2,3,4,5,6,7,8,9,10])
Y = np.array([1,3,6,9,11,13,15,17,20])
m, c = np.polyfit(X, Y, 1)
plt.scatter(X, Y, color='blue', label='Data')
plt.plot(X, m*X + c, color='red', linewidth=2, label='Best Fit Line')
plt.title("Linear Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid(True)
plt.show()
print("Slope =", m)
print("Intercept =", c)
```



```
Slope = 2.333333333333334
Intercept = -3.4444444444444517
```

```
import numpy as np
X = np.array([2, 3, 4, 5, 6, 7, 8, 9, 10])
Y = np.array([1, 3, 6, 9, 11, 13, 15, 17, 20])
x_bar = np.mean(X)
y_bar = np.mean(Y)
numerator = np.sum((X - x_bar) * (Y - y_bar))
denominator = np.sum((X - x_bar) ** 2)
beta1 = numerator / denominator
beta0 = (np.sum(Y) - beta1 * np.sum(X)) / len(X)
print("Mean of X =", np.round(x_bar,2))
print("Mean of Y =", np.round(y_bar,2))
print("Beta1 (Slope) =", np.round(beta1,2))
print("Beta0 (Intercept) =", np.round(beta0,2))
print(f"\nRegression Equation: Y = {beta0:.2f} + {beta1:.2f}X")
```

```
Mean of X = 6.0
Mean of Y = 10.56
Beta1 (Slope) = 2.33
Beta0 (Intercept) = -3.44

Regression Equation: Y = -3.44 + 2.33X
```

```
#Predicting The Value
x=int(input("Enter a value: "))
predicted_ans=-3.44 + 2.33*(x)
print(predicted_ans)
```

```
Enter a value:  2

1.2200000000000002
```

```
#MSE and RMSE of Dataset 1
predicted=[]
errors=0
squarederrors=0
for i in range(len(X)):
    new_value=-3.4444 + 2.3333*(i)
    error=y[i]-new_value
    squarederror=(y[i]-new_value)**2
    squarederrors+=squarederror
    errors+=error
    predicted.append(new_value)

print(round(errors,2))
print(round(squarederrors,2))
```

```
42.0
197.56
```

```
predicted
```

```
[-3.4444,
 -1.1111,
 1.2222,
 3.5555000000000003,
 5.8888,
 8.2221,
 10.5554,
 12.888699999999998,
 15.222]
```

```
df=pd.read_csv('salary_data.csv')
```

```
df.head()
```

```
   YearsExperience   Salary
0              1.1  39343.0
1              1.3  46205.0
2              1.5  37731.0
3              2.0  43525.0
4              2.2  39891.0
```

```
plt.scatter(df['YearsExperience'],df['Salary'])
plt.title("Relation Between Experience and Salary")
plt.xlabel("Year Experience")
plt.ylabel("Salary")
plt.show()
```




```
import numpy as np
x_bar = np.mean(df['YearsExperience'])
y_bar = np.mean(df['Salary'])
numerator = np.sum((df['YearsExperience'] - x_bar) * (df['Salary'] -
y_bar))
denominator = np.sum((df['YearsExperience'] - x_bar) ** 2)
beta1 = numerator / denominator
beta0 = (np.sum(df['Salary']) - beta1 * np.sum(df['YearsExperience']))
/ len(df['YearsExperience'])
print("Mean of X =", np.round(x_bar,2))
print("Mean of Y =", np.round(y_bar,2))
print("Beta1 (Slope) =", np.round(beta1,2))
print("Beta0 (Intercept) =", np.round(beta0,2))
print(f"\nRegression Equation: Y = {beta0:.2f} + {beta1:.2f}X")
```

```
Mean of X = 5.31
Mean of Y = 76003.0
Beta1 (Slope) = 9449.96
Beta0 (Intercept) = 25792.2

Regression Equation: Y = 25792.20 + 9449.96X
```

```
#User Input Dataset 1 using Y = 25792.20 + 9449.96X (B0 and B1)
user_input=float(input("Enter Years of Experience: "))
predicted=25792.20 + 9449.96*user_input
print(predicted)
```

```
Enter Years of Experience:  2.5
```

```
49417.1
```

```
df['Predicted_Salary'] = beta0 + beta1 * df['YearsExperience']
print(df[['YearsExperience', 'Salary', 'Predicted_Salary']])
```

```
    YearsExperience    Salary  Predicted_Salary
0               1.1   39343.0      36187.158752
1               1.3   46205.0      38077.151217
2               1.5   37731.0      39967.143681
3               2.0   43525.0      44692.124842
4               2.2   39891.0      46582.117306
5               2.9   56642.0      53197.090931
6               3.0   60150.0      54142.087163
7               3.2   54445.0      56032.079627
8               3.2   64445.0      56032.079627
9               3.7   57189.0      60757.060788
10              3.9   63218.0      62647.053252
11              4.0   55794.0      63592.049484
12              4.0   56957.0      63592.049484
13              4.1   57081.0      64537.045717
14              4.5   61111.0      68317.030645
15              4.9   67938.0      72097.015574
16              5.1   66029.0      73987.008038
17              5.3   83088.0      75877.000502
18              5.9   81363.0      81546.977895
19              6.0   93940.0      82491.974127
20              6.8   91738.0      90051.943985
21              7.1   98273.0      92886.932681
22              7.9  101302.0     100446.902538
23              8.2  113812.0     103281.891235
24              8.7  109431.0     108006.872395
25              9.0  105582.0     110841.861092
26              9.5  116969.0     115566.842252
27              9.6  112635.0     116511.838485
28             10.3  122391.0     123126.812110
29             10.5  121872.0     125016.804574
```

```
import numpy as np
y_true = df['Salary']
y_pred = df['Predicted_Salary']
mse = np.mean((y_true - y_pred) ** 2)
rmse = np.sqrt(mse)
```

```
print("Mean Squared Error (MSE):", round(mse,2))
print("Root Mean Squared Error (RMSE):", round(rmse,2))
```

```
Mean Squared Error (MSE): 31270951.72
Root Mean Squared Error (RMSE): 5592.04
```

```
comparison = df[['YearsExperience', 'Salary', 'Predicted_Salary']]
comparison = comparison.rename(columns={
    'Salary': 'Actual_Salary',
    'Predicted_Salary': 'Predicted_Salary'
})
print(comparison)
```

```
    YearsExperience  Actual_Salary  Predicted_Salary
0               1.1        39343.0      36187.158752
1               1.3        46205.0      38077.151217
2               1.5        37731.0      39967.143681
3               2.0        43525.0      44692.124842
4               2.2        39891.0      46582.117306
5               2.9        56642.0      53197.090931
6               3.0        60150.0      54142.087163
7               3.2        54445.0      56032.079627
8               3.2        64445.0      56032.079627
9               3.7        57189.0      60757.060788
10              3.9        63218.0      62647.053252
11              4.0        55794.0      63592.049484
12              4.0        56957.0      63592.049484
13              4.1        57081.0      64537.045717
14              4.5        61111.0      68317.030645
15              4.9        67938.0      72097.015574
16              5.1        66029.0      73987.008038
17              5.3        83088.0      75877.000502
18              5.9        81363.0      81546.977895
19              6.0        93940.0      82491.974127
20              6.8        91738.0      90051.943985
21              7.1        98273.0      92886.932681
22              7.9       101302.0     100446.902538
23              8.2       113812.0     103281.891235
24              8.7       109431.0     108006.872395
25              9.0       105582.0     110841.861092
26              9.5       116969.0     115566.842252
27              9.6       112635.0     116511.838485
28             10.3       122391.0     123126.812110
29             10.5       121872.0     125016.804574
```

```
import numpy as np
X = df['YearsExperience'].values
Y = df['Salary'].values
n = len(X)
```

```python
import numpy as np
import matplotlib.pyplot as plt
a0 = 0
a1 = 0
alpha = 0.001
epochs = 10000
cost_history = []
epoch_history = []
for epoch in range(epochs):
    Y_pred = a0 + a1 * X
    cost = (1/n) * np.sum((Y_pred - Y) ** 2)
    cost_history.append(cost)
    epoch_history.append(epoch + 1)
    if (epoch + 1) % 100 == 0:
        print(f"Epoch {epoch+1:5d} | Cost = {cost:.2f}")
    da0 = (2/n) * np.sum(Y_pred - Y)
    da1 = (2/n) * np.sum((Y_pred - Y) * X)
    a0 = a0 - alpha * da0
    a1 = a1 - alpha * da1
print("\nFinal Parameters")
print("Intercept (a0):", round(a0,2))
print("Slope (a1):", round(a1,2))
print(f"Regression Equation: Y = {a0:.2f} + {a1:.2f}X")
```

```
Epoch   100 | Cost = 144494944.21
Epoch   200 | Cost = 135305790.73
Epoch   300 | Cost = 126863847.02
Epoch   400 | Cost = 119106927.81
Epoch   500 | Cost = 111979446.55
Epoch   600 | Cost = 105430327.27
Epoch   700 | Cost = 99412638.58
Epoch   800 | Cost = 93883257.37
Epoch   900 | Cost = 88802559.74
Epoch  1000 | Cost = 84134137.12
Epoch  1100 | Cost = 79844535.29
Epoch  1200 | Cost = 75903014.72
Epoch  1300 | Cost = 72281330.22
Epoch  1400 | Cost = 68953528.60
Epoch  1500 | Cost = 65895762.60
Epoch  1600 | Cost = 63086120.11
Epoch  1700 | Cost = 60504467.05
Epoch  1800 | Cost = 58132303.14
Epoch  1900 | Cost = 55952629.30
Epoch  2000 | Cost = 53949825.87
Epoch  2100 | Cost = 52109540.63
Epoch  2200 | Cost = 50418585.98
Epoch  2300 | Cost = 48864844.44
Epoch  2400 | Cost = 47437181.81
Epoch  2500 | Cost = 46125367.37
Epoch  2600 | Cost = 44920000.57
```

```
Epoch  2700 | Cost = 43812443.69
Epoch  2800 | Cost = 42794759.90
Epoch  2900 | Cost = 41859656.42
Epoch  3000 | Cost = 41000432.24
Epoch  3100 | Cost = 40210930.11
Epoch  3200 | Cost = 39485492.41
Epoch  3300 | Cost = 38818920.61
Epoch  3400 | Cost = 38206438.01
Epoch  3500 | Cost = 37643655.53
Epoch  3600 | Cost = 37126540.23
Epoch  3700 | Cost = 36651386.44
Epoch  3800 | Cost = 36214789.17
Epoch  3900 | Cost = 35813619.73
Epoch  4000 | Cost = 35445003.34
Epoch  4100 | Cost = 35106298.44
Epoch  4200 | Cost = 34795077.88
Epoch  4300 | Cost = 34509111.42
Epoch  4400 | Cost = 34246349.81
Epoch  4500 | Cost = 34004910.08
Epoch  4600 | Cost = 33783062.07
Epoch  4700 | Cost = 33579215.99
Epoch  4800 | Cost = 33391911.07
Epoch  4900 | Cost = 33219805.08
Epoch  5000 | Cost = 33061664.68
Epoch  5100 | Cost = 32916356.65
Epoch  5200 | Cost = 32782839.68
Epoch  5300 | Cost = 32660157.00
Epoch  5400 | Cost = 32547429.45
Epoch  5500 | Cost = 32443849.21
Epoch  5600 | Cost = 32348674.03
Epoch  5700 | Cost = 32261221.88
Epoch  5800 | Cost = 32180866.06
Epoch  5900 | Cost = 32107030.74
Epoch  6000 | Cost = 32039186.81
Epoch  6100 | Cost = 31976848.10
Epoch  6200 | Cost = 31919567.89
Epoch  6300 | Cost = 31866935.70
Epoch  6400 | Cost = 31818574.37
Epoch  6500 | Cost = 31774137.34
Epoch  6600 | Cost = 31733306.17
Epoch  6700 | Cost = 31695788.25
Epoch  6800 | Cost = 31661314.74
Epoch  6900 | Cost = 31629638.60
Epoch  7000 | Cost = 31600532.82
Epoch  7100 | Cost = 31573788.85
Epoch  7200 | Cost = 31549215.02
Epoch  7300 | Cost = 31526635.25
Epoch  7400 | Cost = 31505887.72
Epoch  7500 | Cost = 31486823.76
```

```
Epoch  7600 | Cost = 31469306.74
Epoch  7700 | Cost = 31453211.15
Epoch  7800 | Cost = 31438421.65
Epoch  7900 | Cost = 31424832.24
Epoch  8000 | Cost = 31412345.55
Epoch  8100 | Cost = 31400872.10
Epoch  8200 | Cost = 31390329.67
Epoch  8300 | Cost = 31380642.70
Epoch  8400 | Cost = 31371741.79
Epoch  8500 | Cost = 31363563.14
Epoch  8600 | Cost = 31356048.16
Epoch  8700 | Cost = 31349142.98
Epoch  8800 | Cost = 31342798.12
Epoch  8900 | Cost = 31336968.12
Epoch  9000 | Cost = 31331611.19
Epoch  9100 | Cost = 31326688.96
Epoch  9200 | Cost = 31322166.14
Epoch  9300 | Cost = 31318010.33
Epoch  9400 | Cost = 31314191.74
Epoch  9500 | Cost = 31310683.02
Epoch  9600 | Cost = 31307459.01
Epoch  9700 | Cost = 31304496.61
Epoch  9800 | Cost = 31301774.60
Epoch  9900 | Cost = 31299273.46
Epoch 10000 | Cost = 31296975.29

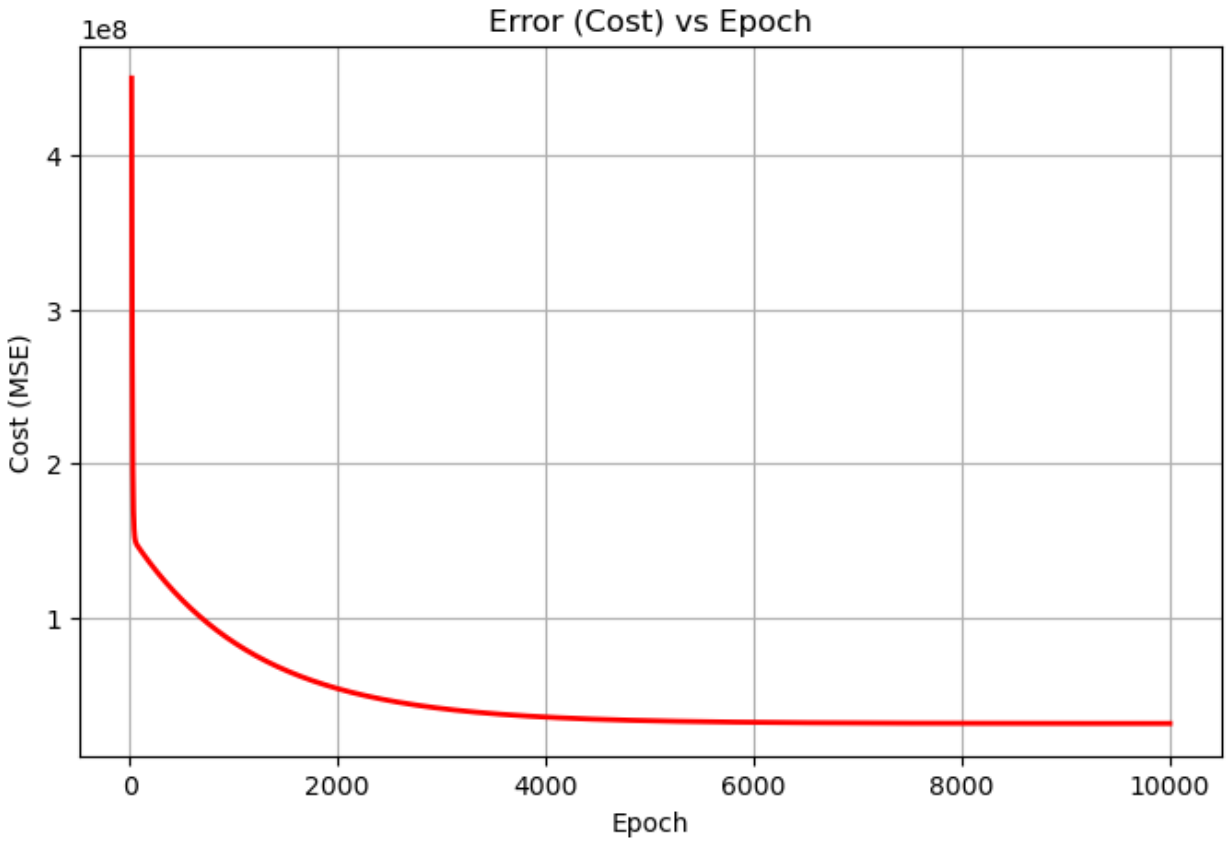
Final Parameters
Intercept (a0): 25445.39
Slope (a1): 9501.43
Regression Equation: Y = 25445.39 + 9501.43X
```

```
#User Input Using Gradient Descent Equation Y = 25445.3913 +
9501.4273X
user_input_gd=float(input("Enter Experience: "))
predicted_gd=25445.3913 + 9501.4273*(user_input_gd)
print(predicted_gd)
```

```
Enter Experience:  2.5

49198.95955
```

```
plt.figure(figsize=(8,5))
plt.plot(epoch_history[20:], cost_history[20:], color='red',
linewidth=2)
plt.title("Error (Cost) vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Cost (MSE)")
plt.grid(True)
plt.show()
```




```
# Predict Salary using Gradient Descent model
df['Predicted_Salary_GD'] = a0 + a1 * df['YearsExperience']
print(df[['YearsExperience', 'Salary',
'Predicted_Salary_GD','Predicted_Salary']])
```

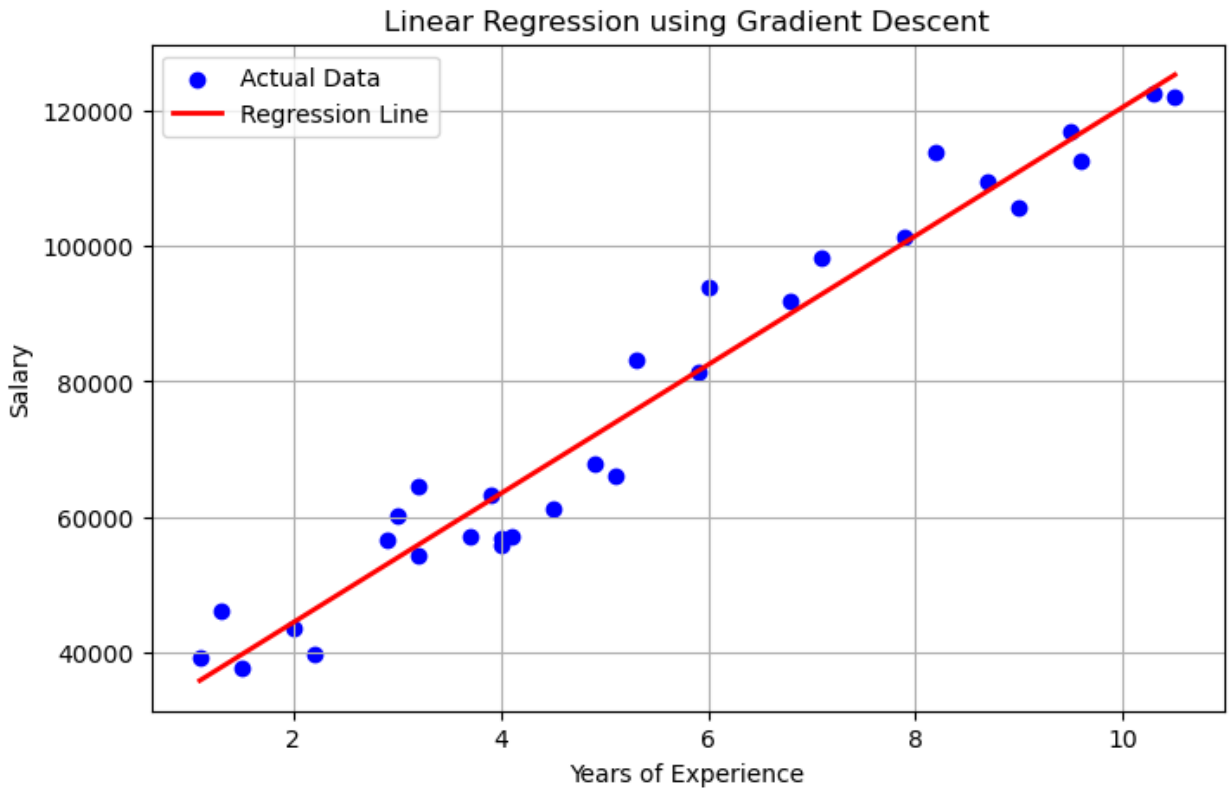
```
    YearsExperience    Salary  Predicted_Salary_GD  Predicted_Salary
0               1.1   39343.0         35896.961324      36187.158752
1               1.3   46205.0         37797.246787      38077.151217
2               1.5   37731.0         39697.532249      39967.143681
3               2.0   43525.0         44448.245906      44692.124842
4               2.2   39891.0         46348.531368      46582.117306
5               2.9   56642.0         52999.530488      53197.090931
6               3.0   60150.0         53949.673219      54142.087163
7               3.2   54445.0         55849.958681      56032.079627
8               3.2   64445.0         55849.958681      56032.079627
9               3.7   57189.0         60600.672338      60757.060788
10              3.9   63218.0         62500.957801      62647.053252
11              4.0   55794.0         63451.100532      63592.049484
12              4.0   56957.0         63451.100532      63592.049484
13              4.1   57081.0         64401.243263      64537.045717
14              4.5   61111.0         68201.814189      68317.030645
15              4.9   67938.0         72002.385114      72097.015574
16              5.1   66029.0         73902.670576      73987.008038
17              5.3   83088.0         75802.956039      75877.000502
```

```
18              5.9   81363.0         81503.812427      81546.977895
19              6.0   93940.0         82453.955158      82491.974127
20              6.8   91738.0         90055.097009      90051.943985
21              7.1   98273.0         92905.525203      92886.932681
22              7.9  101302.0        100506.667053     100446.902538
23              8.2  113812.0        103357.095247     103281.891235
24              8.7  109431.0        108107.808904     108006.872395
25              9.0  105582.0        110958.237098     110841.861092
26              9.5  116969.0        115708.950754     115566.842252
27              9.6  112635.0        116659.093485     116511.838485
28             10.3  122391.0        123310.092605     123126.812110
29             10.5  121872.0        125210.378067     125016.804574
```

```
mse = np.mean((df['Salary'] - df['Predicted_Salary_GD'])**2)
rmse = np.sqrt(mse)
print("Mean Squared Error (MSE):", round(mse,2))
print("Root Mean Squared Error (RMSE):", round(rmse,2))
```

```
Mean Squared Error (MSE): 31296953.27
Root Mean Squared Error (RMSE): 5594.37
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.scatter(df['YearsExperience'], df['Salary'], color='blue',
label='Actual Data')
plt.plot(df['YearsExperience'], df['Predicted_Salary_GD'],
color='red', linewidth=2, label='Regression Line')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Linear Regression using Gradient Descent')
plt.legend()
plt.grid(True)
plt.show()
```




```
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

X=df[['YearsExperience']]
y=df['Salary']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,rando
m_state=42)

lr=LinearRegression()

lr.fit(X_train,y_train)

LinearRegression()

y_pred=lr.predict(X_test)

y_pred

array([115790.21011287,  71498.27809463, 102596.86866063,
75267.80422384,
        55477.79204548,  60189.69970699])

from sklearn.metrics import
mean_absolute_error,mean_squared_error,root_mean_squared_error
print(mean_absolute_error(y_test,y_pred))
```

```
print(mean_squared_error(y_test,y_pred))
print(root_mean_squared_error(y_test,y_pred))
```

```
6286.453830757742
49830096.855908334
7059.043621901506
```

```
lr.coef_
```

```
array([9423.81532303])
```

```
user_input = float(input("Enter Years of Experience: "))
user_df = pd.DataFrame({'YearsExperience': [user_input]})
prediction_lr = lr.predict(user_df)
print("Predicted Salary:", prediction_lr[0])
```

```
Enter Years of Experience:  2.5
```

```
Predicted Salary: 48881.12131935426
```

```
y_pred1=lr.predict(X)
```

```
df['Linear_Model_pred']=y_pred1
```

```
df
```

	YearsExperience	Salary	Predicted_Salary	Predicted_Salary_GD
0	1.1	39343.0	36187.158752	35896.961324
1	1.3	46205.0	38077.151217	37797.246787
2	1.5	37731.0	39967.143681	39697.532249
3	2.0	43525.0	44692.124842	44448.245906
4	2.2	39891.0	46582.117306	46348.531368
5	2.9	56642.0	53197.090931	52999.530488
6	3.0	60150.0	54142.087163	53949.673219
7	3.2	54445.0	56032.079627	55849.958681
8	3.2	64445.0	56032.079627	55849.958681
9	3.7	57189.0	60757.060788	60600.672338
10	3.9	63218.0	62647.053252	62500.957801
11	4.0	55794.0	63592.049484	63451.100532
12	4.0	56957.0	63592.049484	63451.100532

13	4.1	57081.0	64537.045717	64401.243263
14	4.5	61111.0	68317.030645	68201.814189
15	4.9	67938.0	72097.015574	72002.385114
16	5.1	66029.0	73987.008038	73902.670576
17	5.3	83088.0	75877.000502	75802.956039
18	5.9	81363.0	81546.977895	81503.812427
19	6.0	93940.0	82491.974127	82453.955158
20	6.8	91738.0	90051.943985	90055.097009
21	7.1	98273.0	92886.932681	92905.525203
22	7.9	101302.0	100446.902538	100506.667053
23	8.2	113812.0	103281.891235	103357.095247
24	8.7	109431.0	108006.872395	108107.808904
25	9.0	105582.0	110841.861092	110958.237098
26	9.5	116969.0	115566.842252	115708.950754
27	9.6	112635.0	116511.838485	116659.093485
28	10.3	122391.0	123126.812110	123310.092605
29	10.5	121872.0	125016.804574	125210.378067

	Linear_Model_pred
0	35687.779867
1	37572.542932
2	39457.305996
3	44169.213658
4	46053.976722
5	52650.647449
6	53593.028981
7	55477.792045
8	55477.792045
9	60189.699707
10	62074.462772
11	63016.844304
12	63016.844304
13	63959.225836

```
14       67728.751965
15       71498.278095
16       73383.041159
17       75267.804224
18       80922.093418
19       81864.474950
20       89403.527208
21       92230.671805
22       99769.724064
23      102596.868661
24      107308.776322
25      110135.920919
26      114847.828581
27      115790.210113
28      122386.880839
29      124271.643904
```

```
df['differenceb0']=df['Salary']-df['Predicted_Salary']
df['differencegd']=df['Salary']-df['Predicted_Salary_GD']
df['differencelr']=df['Salary']-df['Linear_Model_pred']
```

```
df
```

	YearsExperience	Salary	Predicted_Salary	Predicted_Salary_GD
0	1.1	39343.0	36187.158752	35896.961324
1	1.3	46205.0	38077.151217	37797.246787
2	1.5	37731.0	39967.143681	39697.532249
3	2.0	43525.0	44692.124842	44448.245906
4	2.2	39891.0	46582.117306	46348.531368
5	2.9	56642.0	53197.090931	52999.530488
6	3.0	60150.0	54142.087163	53949.673219
7	3.2	54445.0	56032.079627	55849.958681
8	3.2	64445.0	56032.079627	55849.958681
9	3.7	57189.0	60757.060788	60600.672338
10	3.9	63218.0	62647.053252	62500.957801
11	4.0	55794.0	63592.049484	63451.100532
12	4.0	56957.0	63592.049484	63451.100532

13	4.1	57081.0	64537.045717	64401.243263
14	4.5	61111.0	68317.030645	68201.814189
15	4.9	67938.0	72097.015574	72002.385114
16	5.1	66029.0	73987.008038	73902.670576
17	5.3	83088.0	75877.000502	75802.956039
18	5.9	81363.0	81546.977895	81503.812427
19	6.0	93940.0	82491.974127	82453.955158
20	6.8	91738.0	90051.943985	90055.097009
21	7.1	98273.0	92886.932681	92905.525203
22	7.9	101302.0	100446.902538	100506.667053
23	8.2	113812.0	103281.891235	103357.095247
24	8.7	109431.0	108006.872395	108107.808904
25	9.0	105582.0	110841.861092	110958.237098
26	9.5	116969.0	115566.842252	115708.950754
27	9.6	112635.0	116511.838485	116659.093485
28	10.3	122391.0	123126.812110	123310.092605
29	10.5	121872.0	125016.804574	125210.378067

	Linear_Model_pred	differenceb0	differencegd	differencelr
0	35687.779867	3155.841248	3446.038676	3655.220133
1	37572.542932	8127.848783	8407.753213	8632.457068
2	39457.305996	-2236.143681	-1966.532249	-1726.305996
3	44169.213658	-1167.124842	-923.245906	-644.213658
4	46053.976722	-6691.117306	-6457.531368	-6162.976722
5	52650.647449	3444.909069	3642.469512	3991.352551
6	53593.028981	6007.912837	6200.326781	6556.971019
7	55477.792045	-1587.079627	-1404.958681	-1032.792045
8	55477.792045	8412.920373	8595.041319	8967.207955
9	60189.699707	-3568.060788	-3411.672338	-3000.699707
10	62074.462772	570.946748	717.042199	1143.537228
11	63016.844304	-7798.049484	-7657.100532	-7222.844304
12	63016.844304	-6635.049484	-6494.100532	-6059.844304
13	63959.225836	-7456.045717	-7320.243263	-6878.225836
14	67728.751965	-7206.030645	-7090.814189	-6617.751965

```
15       71498.278095  -4159.015574  -4064.385114  -3560.278095
16       73383.041159  -7958.008038  -7873.670576  -7354.041159
17       75267.804224   7210.999498   7285.043961   7820.195776
18       80922.093418   -183.977895   -140.812427    440.906582
19       81864.474950  11448.025873  11486.044842  12075.525050
20       89403.527208   1686.056015   1682.902991   2334.472792
21       92230.671805   5386.067319   5367.474797   6042.328195
22       99769.724064    855.097462    795.332947   1532.275936
23      102596.868661  10530.108765  10454.904753  11215.131339
24      107308.776322   1424.127605   1323.191096   2122.223678
25      110135.920919  -5259.861092  -5376.237098  -4553.920919
26      114847.828581   1402.157748   1260.049246   2121.171419
27      115790.210113  -3876.838485  -4024.093485  -3155.210113
28      122386.880839   -735.812110   -919.092605      4.119161
29      124271.643904  -3144.804574  -3338.378067  -2399.643904
```

```
df['mseb0']=(df['Salary']-df['Predicted_Salary'])**2
df['msegd']=(df['Salary']-df['Predicted_Salary_GD'])**2
df['mselr']=(df['Salary']-df['Linear_Model_pred'])**2

df
```

```
    YearsExperience    Salary  Predicted_Salary
Predicted_Salary_GD  \
0               1.1   39343.0      36187.158752          35896.961324

1               1.3   46205.0      38077.151217          37797.246787

2               1.5   37731.0      39967.143681          39697.532249

3               2.0   43525.0      44692.124842          44448.245906

4               2.2   39891.0      46582.117306          46348.531368

5               2.9   56642.0      53197.090931          52999.530488

6               3.0   60150.0      54142.087163          53949.673219

7               3.2   54445.0      56032.079627          55849.958681

8               3.2   64445.0      56032.079627          55849.958681

9               3.7   57189.0      60757.060788          60600.672338

10              3.9   63218.0      62647.053252          62500.957801

11              4.0   55794.0      63592.049484          63451.100532

12              4.0   56957.0      63592.049484          63451.100532

13              4.1   57081.0      64537.045717          64401.243263
```

```
14              4.5   61111.0      68317.030645          68201.814189
15              4.9   67938.0      72097.015574          72002.385114
16              5.1   66029.0      73987.008038          73902.670576
17              5.3   83088.0      75877.000502          75802.956039
18              5.9   81363.0      81546.977895          81503.812427
19              6.0   93940.0      82491.974127          82453.955158
20              6.8   91738.0      90051.943985          90055.097009
21              7.1   98273.0      92886.932681          92905.525203
22              7.9  101302.0     100446.902538         100506.667053
23              8.2  113812.0     103281.891235         103357.095247
24              8.7  109431.0     108006.872395         108107.808904
25              9.0  105582.0     110841.861092         110958.237098
26              9.5  116969.0     115566.842252         115708.950754
27              9.6  112635.0     116511.838485         116659.093485
28             10.3  122391.0     123126.812110         123310.092605
29             10.5  121872.0     125016.804574         125210.378067


    Linear_Model_pred  differenceb0  differencegd  differencelr
mseb0  \
0        35687.779867   3155.841248   3446.038676   3655.220133
9.959334e+06
1        37572.542932   8127.848783   8407.753213   8632.457068
6.606193e+07
2        39457.305996  -2236.143681  -1966.532249  -1726.305996
5.000339e+06
3        44169.213658  -1167.124842   -923.245906   -644.213658
1.362180e+06
4        46053.976722  -6691.117306  -6457.531368  -6162.976722
4.477105e+07
5        52650.647449   3444.909069   3642.469512   3991.352551
1.186740e+07
6        53593.028981   6007.912837   6200.326781   6556.971019
3.609502e+07
7        55477.792045  -1587.079627  -1404.958681  -1032.792045
```

```
2.518822e+06
8        55477.792045   8412.920373   8595.041319   8967.207955
7.077723e+07
9        60189.699707  -3568.060788  -3411.672338  -3000.699707
1.273106e+07
10       62074.462772    570.946748    717.042199   1143.537228
3.259802e+05
11       63016.844304  -7798.049484  -7657.100532  -7222.844304
6.080958e+07
12       63016.844304  -6635.049484  -6494.100532  -6059.844304
4.402388e+07
13       63959.225836  -7456.045717  -7320.243263  -6878.225836
5.559262e+07
14       67728.751965  -7206.030645  -7090.814189  -6617.751965
5.192688e+07
15       71498.278095  -4159.015574  -4064.385114  -3560.278095
1.729741e+07
16       73383.041159  -7958.008038  -7873.670576  -7354.041159
6.332989e+07
17       75267.804224   7210.999498   7285.043961   7820.195776
5.199851e+07
18       80922.093418   -183.977895   -140.812427    440.906582
3.384787e+04
19       81864.474950  11448.025873  11486.044842  12075.525050
1.310573e+08
20       89403.527208   1686.056015   1682.902991   2334.472792
2.842785e+06
21       92230.671805   5386.067319   5367.474797   6042.328195
2.900972e+07
22       99769.724064    855.097462    795.332947   1532.275936
7.311917e+05
23      102596.868661  10530.108765  10454.904753  11215.131339
1.108832e+08
24      107308.776322   1424.127605   1323.191096   2122.223678
2.028139e+06
25      110135.920919  -5259.861092  -5376.237098  -4553.920919
2.766614e+07
26      114847.828581   1402.157748   1260.049246   2121.171419
1.966046e+06
27      115790.210113  -3876.838485  -4024.093485  -3155.210113
1.502988e+07
28      122386.880839   -735.812110   -919.092605      4.119161
5.414195e+05
29      124271.643904  -3144.804574  -3338.378067  -2399.643904
9.889796e+06

           msegd         mselr
0   1.187518e+07  1.336063e+07
1   7.069031e+07  7.451932e+07
```

```
2   3.867249e+06  2.980132e+06
3   8.523830e+05  4.150112e+05
4   4.169971e+07  3.798228e+07
5   1.326758e+07  1.593090e+07
6   3.844405e+07  4.299387e+07
7   1.973909e+06  1.066659e+06
8   7.387474e+07  8.041082e+07
9   1.163951e+07  9.004199e+06
10  5.141495e+05  1.307677e+06
11  5.863119e+07  5.216948e+07
12  4.217334e+07  3.672171e+07
13  5.358596e+07  4.730999e+07
14  5.027965e+07  4.379464e+07
15  1.651923e+07  1.267558e+07
16  6.199469e+07  5.408192e+07
17  5.307187e+07  6.115546e+07
18  1.982814e+04  1.943986e+05
19  1.319292e+08  1.458183e+08
20  2.832162e+06  5.449763e+06
21  2.880979e+07  3.650973e+07
22  6.325545e+05  2.347870e+06
23  1.093050e+08  1.257792e+08
24  1.750835e+06  4.503833e+06
25  2.890393e+07  2.073820e+07
26  1.587724e+06  4.499368e+06
27  1.619333e+07  9.955351e+06
28  8.447312e+05  1.696749e+01
29  1.114477e+07  5.758291e+06
```

```
print(f"MAE of B0 Model: {np.mean(abs(df['differenceb0'])):.2f}")
print(f"MAE of GD Model: {np.mean(abs(df['differencegd'])):.2f}")
print(f"MAE of Linear Regression Model:
{np.mean(abs(df['differencelr'])):.2f}")
print("-----------------------------------------------")
print(f"MSE of B0 Model: {np.mean(abs(df['mseb0'])):.2f}")
print(f"MSE of GD Model: {np.mean(abs(df['msegd'])):.2f}")
print(f"MSE of Linear Regression Model:
{np.mean(abs(df['mselr'])):.2f}")
```

```
MAE of B0 Model: 4644.20
MAE of GD Model: 4637.55
MAE of Linear Regression Model: 4634.13
-----------------------------------------------
MSE of B0 Model: 31270951.72
MSE of GD Model: 31296953.27
MSE of Linear Regression Model: 31647819.16
```